Department of Computer Science, University College London

COMP0227 Coursework 1: Monocular Depth in Monocular SLAM

Due Friday 13th November, 2026, 16:00

1 Scope and Purpose

A pure monocular SLAM system can only estimate the depth to a feature point by using motion parallax and triangulation in two or more views. This has well-known consequences. The map has no metric scale, and the scale drifts over a long sequence. Triangulation is poor when the camera rotates more than it translates, because there is too little parallax.

Monocular depth networks give every pixel of a single image a depth. Such a depth is available from the first frame, needs no parallax, and may even be metric. This can add stability to the system, especially in scenarios where triangulation is poor. But it is learned: it can be wrong at the edges of objects, on surfaces unlike those the network was trained on, and it can change from one frame to the next even when the scene does not.

In this coursework you will combine the two. Starting from pySLAM, you will feed the depth predicted by a monocular depth network into monocular SLAM in several different ways, measure whether each one helps or hurts, and explain why. The question is:

When does learned depth help a monocular SLAM system?

There is no single right method, and a sound investigation whose answer is that learned depth does not help on these sequences is a valid result. The marks reward a fair baseline, a careful measurement, several methods compared under the same conditions, an explanation of the results, and a clear report that others can reproduce.

You work from the tagged course version of pySLAM, which we will name when the coursework is released.

2 The Coursework at a Glance

The coursework is five parts. Each builds on the one before, and each is marked on its own, so a group that does Parts 1 to 3 well earns those marks even if Part 4 is thin. Marks are out of 100 for the coursework, which is 35% of the module.

The Five Parts

Part 1	→	Part 2	→	Part 3	→	Part 4	→	Part 5
Baseline and measurement		Depth from one image and feature points		Depth in SLAM		Agreement and choice of frames		Report and reproducibility
10		15		20		30		25

Figure 1: The five parts and their marks. Each part uses what the one before produced: the sequences, the baseline and the measuring script (Part 1); a depth map for every frame, and a depth, an uncertainty and a scale at the keypoints (Part 2); your best method (Part 3).

Part 1. Baseline and measurement (10 marks). *How well does monocular SLAM do on these sequences, and how much do two runs differ?*
You get the three sequences, KITTI 06, TUM fr3 long office household and Replica `room0`, into a form pySLAM can run, run pySLAM unchanged, and run it with real depth where a sequence has some, as a reference. You write the one script that measures every run from then on. Everything later is compared with this baseline, and a difference smaller than its run-to-run spread does not count.

Part 2. Depth from one image and feature points (15 marks). *How good is the learned depth on its own, before SLAM sees it?*
You run one network on every frame and save the depth maps, which every later part reuses. You score them against the true depth where a sequence has it, and measure how the depth of the same surface changes from frame to frame. SLAM only looks at its keypoints, so you read the depth there, give each value an uncertainty, and bring it to the scale of the map. These, a depth and an uncertainty for each keypoint and a scale for the map, are what the methods of Part 3 consume.

Part 3. Depth in SLAM (20 marks). *Which way of putting learned depth into SLAM helps, and why?*
You run the simplest one, treating the depth as if it came from a depth camera, as a reference, and design at least two methods of your own. You run each under the conditions of Part 1, and vary how much each one trusts the depth.

Part 4. Agreement and choice of frames (30 marks, of which 15 for your own sequence). *When is learned depth better than triangulation, and when does it make things worse?*
You compare the two depths of the same map points, check that a point's learned depth agrees across keyframes, and decide which frames need depth at all. You find one case where depth hurts and trace the cause. Finally you record a sequence of your own where you expect depth to help or hurt, predict what will happen, and test the prediction. Show evidence such as ATE, RPE and other measures to support your answer.

Part 5. Report and reproducibility (25 marks). *Can you answer the question of Section 1 with your own numbers, so that someone else can check them?*
These marks are for the quality and clarity of the report and for the reproducibility of your results; the results themselves are marked in Parts 1 to 4. You write a 5-page IEEE paper which leads with the answer and explains it by what each method does. You hand in the code and the results of every run, so that every table can be rebuilt.

Sections 3 to 7 say what we would like you to do in each part. Sections 8 to 10 give what you need for every part: the submission, the software and the sequences, and the measures.

3 Part 1: Baseline and Measurement (10 marks)

In this part you will develop the tools and establish the baseline performance of the monocular SLAM system. We will use three sequences: KITTI 06, TUM fr3 long office household and Replica `room0` (Section 9). You will need to start by getting KITTI 06 and Replica `room0` running in monocular mode in pySLAM, with their ground-truth trajectories; the TUM sequence is already in the form pySLAM reads. You will also need the script which computes the measures of Section 10 for any run, since you will use it on every run from now on. You should compare the performance of pySLAM's C++ and Python cores. Your baseline should include running and analysing the measures of Section 10. You should also investigate pySLAM's determinism: if you run the same sequence through several times, do you get the same trajectory or not, and what does that mean for how many runs you need before you believe a difference between two methods? TUM and Replica also have a depth image for every frame, so you can run pySLAM in RGB-D mode on them as a reference for the best that depth could do.

4 Part 2: Depth from One Image and Feature Points (15 marks)

In this part you will produce the learned depth which the rest of the coursework uses, and find out how far it can be trusted. Choose one of the three networks of the `depth` level and run it on every frame of the three sequences, saving the depth maps so that pySLAM can read them as the depth images of an RGB-D sequence. You should then assess the depth on its own, before SLAM sees it. Replica `room0` and the TUM sequence have depth images, exact on Replica and from a real depth camera on TUM, so you can score the predictions against them, both as the network gives them and with the scale removed; KITTI 06 has none, so you will have to think about what you can still say about the network there. Look at where the errors are: is there a pattern to the regions of high and low error, and is there anything you could do about it? You should also look at how the depth of a surface which does not move changes from one frame to the next. Finally, SLAM only uses the depth at its keypoints, so you should look at the depth there in particular: how to read it from the depth map, how uncertain it is, and how to bring it to the scale of the map. By the end of this part you should have a depth and an uncertainty for every keypoint, and a scale for the map.

5 Part 3: Depth in SLAM (20 marks)

In this part you will put the learned depth into monocular SLAM. The simplest way is to give pySLAM the predicted depth as if it came from a depth camera, which needs no change to pySLAM; run this first, as your reference. You should then design and implement at least two further ways of using the depth, which may change either pySLAM's C++ core or its Python core. Where in SLAM they act, and how, is your choice, but for each you should be able to say where the depth enters, whether it is a starting point which the optimisation may abandon or a constraint which stays, which source wins when a point has both a triangulated and a learned depth, and whether the map ends up in metres. Run every method under the same conditions as your baseline of Part 1 and measure it with the same tools, so that all the methods, the baseline and the reference can be compared in one table; say what each method is compared with, and why that comparison is fair. Each method should have at least one setting which decides how far the learned depth is trusted; vary it and show what happens, rather than only reporting the best value.

6 Part 4: Agreement and Choice of Frames (30 marks)

In this part you will find out when the learned depth is better than triangulation and when it makes things worse. Using the maps your methods built, you should compare the triangulated depth of the map points with their learned depth, and look at how the disagreement depends on the parallax, the distance and where the point is in the image. You should check whether a map point's learned depth agrees across the keyframes which observe it, and decide what to do with the points where it does not. You should also investigate which frames need depth at all, and what each choice costs in time. Find at least one case where the learned depth makes SLAM worse than the baseline, and trace it from what the network got wrong to what it did to the trajectory.

Finally (15 of the 30 marks), record a sequence of your own, with a calibrated phone camera or the module's RealSense D455, in a scene where you expect learned depth to help or to hurt. Before you record, write down what you expect your best method to do compared with the baseline, and why. Then run both on it and test your prediction, with the same measures as for the other sequences. There is no ground-truth trajectory, so you will need a reference one: a reconstruction of the sequence by a separate method, for example structure from motion with COLMAP or a feed-forward model such as VGGT, brought to metres with a distance you measured in the scene, is the usual way. Say how far this reference can be trusted in your scene: it is built from the same images, and it either triangulates, which is what your methods are meant to help with, or relies on a learned prior of its own.

7 Part 5: Report and Reproducibility (25 marks)

The report brings the parts together. The marks of this part are for the quality and clarity of the report and for the reproducibility of your results, since the results themselves are marked in Parts 1 to 4. It is a paper in the IEEE conference format of at most five pages (Section 8). It should lead with the answer to the question of Section 1, supported by your own numbers, and explain the results by what each method does rather than only ranking them; the failures belong in it, with their causes. With it you hand in your code and the results of every run, so that another group could rebuild every table from your `README`.

8 Organisation and Submission

Groups

You work in groups, which you choose yourselves. By default, all the members of a group receive the same mark. If you run into problems working together, tell us as soon as possible; if you cannot agree on the contribution statement, speak to the module lead or a TA at least a week before the deadline.

What You Hand In

Each group submits the items of Table 1 on Moodle by 16:00 on Friday 13th November, 2026.

The report should start with the answer: which method works best, by how much, on which sequences, and why. Your numbers must come from your own runs. Numbers taken from papers may be quoted for comparison, labelled as such.

Item	Content
Report (PDF)	A paper in the IEEE conference format (`https://www.ieee.org/conferences/publishing/templates.html`), at most 5 pages excluding references. The method and results of every part, and what you conclude. It ends with a contribution statement, which says explicitly whether all the members contributed equally and, if not, what each member contributed.
Code (ZIP)	One `zip` archive, not a link to a repository: your changes to pySLAM as a `git diff` against the tagged commit you started from, the scripts which produce your depth maps, run every configuration and make every table and figure of the report, and a `README` which says which command makes which table. Leave out pySLAM itself, the sequences and the network weights.
Results (CSV)	One table with one row for each run and a column for each of: the sequence (KITTI 06, fr3, room0, or your own), the method and its settings, the run number, and each measure of Section 10 that you report (the ATE without and with a fitted scale, the scale, the RPE, the shares of frames tracked and lost, the time per frame). All the runs, not only the averages. With it, the estimated trajectory of every run in the TUM format, so that every error can be recomputed.

Table 1: Items to hand in.

9 Setting Up

The Depth Level

The depth networks need the `depth` level of pySLAM, a second pixi environment, which has three monocular networks: Depth Anything V2, Depth Anything 3 and Depth Pro, all trained to give metres. Install it and their weights with:

```
pixi run -e depth models-depth
```

This is a large download, about 7 GB of weights. The SLAM runs themselves stay in the default environment.

The Sequences

Parts 1 to 3 use the same three sequences for every group, so that the results of different groups can be compared. All three have a ground-truth trajectory. They differ in how much you know about the depth, from everything to nothing (Table 2).

KITTI 06 is the realistic case: a robot with one camera and no depth sensor. There, the only way to tell whether the learned depth is right is through what it does to SLAM.

The TUM sequence is downloaded from the TUM RGB-D benchmark web page as `rgbd_dataset_freiburg3_long_office_household`. KITTI 06 is in `data/videos/kitti06/`. Replica `room0` is one of the eight scenes of the Replica package which pySLAM's README links (12 GB for all eight).

Your own sequence comes in at the end, in Part 4: you record it once you know from Part 3 what to look for. Record it with a calibrated phone camera, or with the module's RealSense D455, which also records a depth image with every frame: ask the module lead for the camera.

Running the Sequences

pySLAM's command line reads a video, a folder of images, or a sequence in the layout of the TUM RGB-D benchmark, as monocular SLAM or with a depth image for every frame (`pixi run slam -help`

Sequence	Settings file	Depth ground truth	Unique properties
Replica `room0`	`REPLICA.yaml`	**perfect: exact depth for every pixel of every frame**	**a rendered indoor room, synthetic and without noise: errors come from the method, not the data**
TUM RGB-D fr3 long office household	`TUM3.yaml`	**a real depth camera: noisy, with holes, unreliable beyond a few metres**	**indoors, hand-held, a long loop around an office**
KITTI 06	`KITTI04-12.yaml`	**none**	**outdoors, fast forward motion, depths of tens of metres; the colour video which comes with pySLAM**

Table 2: The three common sequences. The settings files are in `settings/`**.**

lists the options). The TUM sequence can be run as it comes. How you get KITTI 06 and Replica `room0` **running, with their ground truth, is yours to work out (Part 1).**

A run with a sequence's own depth images shows the best that depth can do; keep it as a reference. A run in which the depth predicted by a network takes the place of the camera's depth is *pseudo RGB-D*, the reference method of Part 3.

10 Measuring

Two runs of the same configuration do not give the same numbers, so a comparison between two methods has to show that their difference is larger than the variation between runs of the same one. How many runs that takes is for you to find out in Part 1; report the spread as well as the central value, and say what you report.

For every configuration we need to see: the error of the trajectory against the ground truth (ATE), both with and without a fitted scale, and the scale that was fitted, so that monocular runs and runs which use depth can be compared on the same footing; the drift over a fixed interval (RPE); how much of the sequence was tracked, and where tracking was lost; where the sequence has depth, how well the map points agree with it; and the time per frame, with the computer you used. How you compute these is up to you, but say exactly what you computed.

11 Rules

- **You may use any public depth network and any open-source code, cited in the report. Pretrained weights are fine: you are not expected to train a network.**
- **This assessment is in Category 3 of UCL's three categories of generative-AI use in assessment, in which AI tools have an integral role:** `https://www.ucl.ac.uk/teaching-learning/generative-ai-hub/three-categories-genai-use-assessment`**. Say in the report where you used them. You will have to explain your approach in the viva later in the module, so use AI responsibly: know what you did, and why you did it.**
- **Your changes to pySLAM must not be pushed to a public repository before the deadline.**
- **Any undeclared use of AI, in particular in writing the report or generating code, will be treated as academic misconduct.**